

Chat History - Traffic Escape

Software Engineering Lab 4 - Vibe Coding Assignment (Python / Pygame)

Contents: Part 1 Codebase inspection | Part 2 Task 1 Lives System | Part 3 Task 2 Moving Log/Raft Lane | Part 4 Task 3 High Score Table | Part 5 Task 4 Day/Night Cycle | Appendix A files per task | Appendix B final source code

Name: _____ SRN / Roll No: _____ Section: _____

Course: Software Engineering - Lab 4 (Vibe Coding) Tool used: Claude (Anthropic) Date: 07 October 2026

Yellow boxes mark messages that were not in the material provided for this PDF (the Task 2 prompt and reply, and the Task 3 prompt). Everything else is the original conversation text. Paste the missing messages from your original chat into those places before submitting if your lab needs the full raw log.

Part 1 - Codebase inspection

User

I am completing my Software Engineering Lab 4 Vibe Coding assignment.

Project: Traffic Escape

Language: Python / Pygame

You have the complete current project files and README.

First, inspect and understand the existing codebase and architecture.

Important rules:

- Do not rewrite the entire project.
- Preserve the existing gameplay, controls, scoring, lane system and overall design unless a task specifically requires a change.
- We will implement exactly four tasks, one at a time.
- Do not implement future tasks early.
- After each task, I will test the program locally and create a separate Git commit.
- Make minimal, clean, understandable changes.
- Tell me which files need to be changed before making the changes.

The four required tasks from the README are:

- Task 1: Lives System
- Task 2: Moving Log/Raft Lane
- Task 3: High Score Table
- Task 4: Day/Night Cycle

For now, ONLY inspect the project and explain:

- the current architecture
- what is currently broken or missing
- which files are likely to be modified for Task 1
- any potential risks when implementing Task 1

Do not implement anything yet.

Claude

The file contents are already in context, so I read them directly. Here is my analysis, with no code changes.

Current architecture

Entry point: main.py creates a GameEngine and calls run().

game_engine.py (GameEngine) owns the whole game loop and state:

- reset() builds a fresh Player, an empty car list, the spawn timer/interval, speed, score, and the game_over / won flags.
- handle_events() handles quit and the R key (calls reset()).
- update() moves the player, spawns cars at random lanes, updates cars, checks collisions, increments the score, ramps up speed, and checks the win condition (player.rect.top <= 10).
- draw() renders the road, sidewalks, cars, player, HUD, and the CRASHED / YOU MADE IT overlays.

player.py (Player) has a rect, a lane-snapping horizontal move (with a 12-frame cooldown) and continuous vertical movement (8 px/frame).

traffic.py has Car (rect, direction, speed) and make_car(), which picks direction by lane parity (even lanes go down, odd lanes go up).

What is broken or missing

Bugs and inconsistencies I noticed (the README mentions a "bug" to show in the "before" video):

- Import structure doesn't match the code. The imports use from game.game_engine import ... and from game.player import ..., so the files must live in a game/ package (as the README's folder structure says). The uploaded files are flat, so you need to arrange them into game/ before it runs.
- Instant death on spawn: make_car() spawns cars at y=-90 (or height+10) in a random lane with no check for the player's position. Not usually a problem, but cars spawn directly on top of the player's lane with no warning.
- HUD overlap: the HUD bar covers the top 30px, which is also the top sidewalk and the win zone (rect.top <= 10), so the player is drawn under or behind the HUD at the finish.
- Spawn interval float drift: spawn_interval becomes a float (-0.2 per spawn) and is compared against an int timer. It works, but it's slightly sloppy.
- Score/speed coupling: if self.score%300==0 works only because score increments by exactly 1 per frame. It's fragile if the score ever changes in other ways (like bonuses).
- Win-state score keeps no record, and the score is a survival timer rather than a result, which matters for Task 3 later.
- Moving backward is allowed into the bottom sidewalk area and beyond: ny=max(0,...) clamps only the top, so the player can drive off the bottom of the screen.

Missing features (the four tasks): lives, log/raft lane, high-score persistence, and day/night cycle. None exist yet.

Task 1 (Lives System): likely files to modify

- game_engine.py (main change): add a lives attribute (3) in reset(), replace the instant game_over=True on collision with "lose a life", and show lives in the HUD. Game over triggers only at 0.
- player.py (small change, probably): a way to reset the player's position to the start after a hit, and possibly brief invulnerability frames.
- traffic.py: no change expected.
- main.py, requirements.txt, __init__.py: no change.

Risks for Task 1

- Repeated hits on one collision. Collision is checked every frame at 60 FPS, so one car overlapping the player would drain all 3 lives in about 3 frames. This is the main risk. A short invulnerability window (about 1 to 2 seconds) or a respawn that clears nearby cars is needed.

- Respawn location. If the player respawns at the start while a car is passing there, they are hit again immediately. Invulnerability or clearing nearby cars solves this.
- Restart (R key) must also reset lives, which works automatically if lives is set inside reset().
- HUD space. The HUD text is already long ("Score... GOAL... R=Restart") on a 640px-wide screen, so adding lives may overflow. I'd shorten the text or put lives in a separate spot.
- Game-over vs. win states. The update() early return and the overlays should keep working unchanged. Losing a life must not set game_over until lives reach 0.
- Scope creep. Invulnerability flashing is visual polish. I'd keep it minimal (a simple blink) so it doesn't leak into later tasks.

Proposed plan for Task 1

I would modify only game_engine.py and player.py, with a few lines each: a lives counter, a hit_cooldown (invulnerability frames), a Player.respawn() method, and the HUD update. Say the word and I'll implement it.

Part 2 - Task 1: Lives System

User

Now implement Task 1 ONLY: Lives System.

Requirements:

- The player starts with exactly 3 lives.
- When the player is hit by a car, exactly 1 life is lost.
- The game must NOT immediately end after the first collision.
- If lives remain, the player should be reset/repositioned appropriately and gameplay should continue.
- Game over must occur only when the number of lives reaches 0.
- Display the current lives clearly in the existing HUD/interface.
- Preserve the existing controls.
- Preserve the existing score behavior.
- Preserve the existing traffic behavior.
- Preserve the existing win condition.
- Do not implement Task 2, Task 3 or Task 4 yet.
- Avoid unnecessary architectural changes.
- Handle collision/reset state correctly so one collision does not repeatedly remove multiple lives from a single event.
- Implement the changes in the existing project files.

After making the changes:

- Explain what you changed.
- List every modified file.
- Explain how the collision and life-reset logic works.
- Check for obvious runtime errors or logic issues.
- Do not modify anything unrelated to Task 1.
- Fitting lives and restart text into the HUD layout.

Claude

I'll set up the game/ package layout the imports expect, make the Task 1 edits, then test headlessly.

Process notes: edited 2 files and ran several commands. My Python rewrite normalised the line endings: the original files use CRLF, so Git would show the whole file as changed. I restored CRLF so the diff only shows the real edits.

Task 1 (Lives System) is implemented, and I tested it headlessly with pygame 2.5.2. I haven't run it with a visible window, so please play it locally before committing.

1. What I changed

game/game_engine.py

- Added two constants: `START_LIVES = 3` and `INVULN_FRAMES = 90` (about 1.5 seconds at 60 FPS).
- In `reset()`: added `self.start_pos`, `self.lives = START_LIVES` and `self.invuln = 0`. Because they live in `reset()`, the R key restores lives automatically.
- In `update()`: the old "`collision means game_over = True`" is replaced by a call to a new `lose_life()` method.
- Added `lose_life()`: it subtracts 1 life. At 0 it sets `game_over`; otherwise it respawns the player and starts the invulnerability timer.

- In `draw()`: the player blinks while invulnerable, and the HUD shows Lives: N in red, right-aligned.

game/player.py

- Added `respawn(x, y)`. It moves the player back to the start centre and clears `move_cooldown`.

2. Modified files

Only game/game_engine.py and game/player.py. main.py, traffic.py, __init__.py, requirements.txt and README.md are byte-identical to what you uploaded.

Your uploads were flat, but the imports expect a game/ package, so I arranged them as the README's folder structure shows. Both files keep their original CRLF line endings, so git diff shows only the real changes.

3. How the collision and reset logic works

- Each frame, `invuln` counts down if it's above 0, then all cars move.
- A collision is checked only when `invuln == 0`, and it checks all cars at once. Two cars hitting on the same frame cost 1 life, not 2.
- On a hit, `lose_life()` subtracts one life.
- If lives are still above 0, the player returns to the start position and `invuln` is set to 90 frames. Collisions are ignored during that time, so the car that hit you can't hit you again every frame.
- If lives reach 0, `game_over = True`, and the existing "CRASHED!" overlay and the early return in `update()` handle the rest.
- The win check, score increment and speed ramp are untouched. Score keeps counting through life losses.

4. Checks I ran

- A single hit costs exactly 1 life, the game continues, and the player respawns at the start.
- A stationary car sitting on the respawn spot costs no extra life during the 90-frame window. It does hit again once the window ends, which is the correct behaviour.
- Two cars overlapping the player on one frame cost 1 life.
- Game over triggers exactly when lives hit 0, and the game freezes afterwards.
- R restores 3 lives, and the win condition still works.
- A 20,000-frame random soak ran with no exceptions, and lives stayed between 0 and 3.

Things to know

- HUD text changed slightly. The original HUD text was already 616px of the 640px window, so there was no room for lives. I shortened "GOAL: reach the top!" to "GOAL: top!", which fits with room to spare. If you'd rather keep the original wording, the alternative is a smaller font.
- Cars are not cleared on respawn. The invulnerability window covers this. Cars move 3 to 10 px/frame, so anything overlapping the start area passes within the 1.5 seconds.
- Existing bugs are untouched. The earlier issues I listed (player can drive off the bottom, HUD overlapping the finish, and so on) are still there. Those are not Task 1.

For your "before" and "after" videos, the old behaviour is a single crash ending the game. Now a crash makes the car blink back at the start and the counter drops from 3 to 2.

Attached files: game_engine.py, player.py

Part 3 - Task 2: Moving Log/Raft Lane

User

MISSING: the Task 2 prompt was not in the material provided. Paste it here from your original chat.

Claude

MISSING: the original Task 2 reply was not in the material provided. Paste it here from your original chat.

Summary from the submitted code (not the original reply)

This section is NOT the original Claude reply. It is a short description written from the submitted code (river.py and game_engine.py) because the original Task 2 messages were not in the material provided.

- New file game/river.py: a River class holding a water band (RIVER_TOP = 250, RIVER_H = 80) and 3 Log objects (140 px wide) that move right at 2 px per frame and wrap around the screen edge.
- River.contains(point) tells whether a point is over the water band, and River.log_at(point) returns the log under that point, or None.
- game_engine.py: reset() creates self.river = River(WIDTH); draw() draws the river over the cars so they pass underneath it.
- update_river(): looks up the log under the player's centre, moves the logs, then moves the player with that log (clamped inside the screen).
- If the player's centre is over the water and no log is under it, the player has fallen in and lose_life() is called, so it costs exactly one life and uses the Task 1 respawn.
- car_hits_player(): overlap that lies entirely inside the river band is ignored, because cars run underneath the river.

Part 4 - Task 3: High Score Table

User

MISSING: the Task 3 prompt was not in the material provided. Paste it here from your original chat.

Claude

Process notes: Aligning high-score rows with consistent spacing and rank markers. Task 3 touches the end-of-run flow, so I'll check the Task 2 state first, snapshot it for diffing, and confirm which code paths end a run. The diff is small and the other files are unchanged. I'll test in a throwaway copy of the project so no highscores.json ends up in your tree. This also checks the real on-disk location.

Task 3 (the high score table) is implemented and passes my headless tests, but I haven't played it in a visible window. I did render the game-over and win screens to check how they look.

1. Files modified

- game/highscores.py (new): `load_scores()`, `save_scores()` and `add_score()`, with the file path and the top-5 limit.
- game/game_engine.py: about 30 added lines: an import, loading at start-up, a `finish_run()` method, and a `_draw_scores()` method for the table.

Every other file is byte-identical to the Task 2 state. The new file uses CRLF like your others.

2. Where the JSON file is stored

The scores are in `highscores.json` in the project root, next to `main.py`. The path is built from the module's own location, so the game finds the same file no matter which folder you start it from. I tested launching from `/`.

The file is a plain list, for example `[412, 305, 288, 190, 64]`. It doesn't exist until the first run ends. If you don't want it in your commits, add it to `.gitignore`.

3. How scores are loaded and saved

Loading happens once, in `GameEngine.__init__`, before `reset()`. That means pressing R doesn't reload or lose the table. `load_scores()` never raises an exception:

- A missing, unreadable or invalid file gives an empty list, so the first run just works.
- A file that isn't a list (such as `{}` or `null`) also gives an empty list.
- A list with bad entries (strings, null, true, negatives, decimals) keeps only the valid whole numbers, sorted highest first and trimmed to 5.

Saving writes to `highscores.json.tmp` and then swaps it in with `os.replace()`. An interrupted write can't corrupt the existing file. If saving fails, for example in a read-only folder, the game prints one warning line and carries on, and the table still works in memory for that session.

4. When the table is updated

Both ways a run ends (game over at 0 lives, or reaching the top) happen inside `update()`, which returns early on every later frame. A check at the end of `update()` therefore fires exactly once per run. A test confirms 500 extra frames add no duplicates.

- What gets recorded: `finish_run()` records the same number the HUD shows (`score // 10`), so the existing scoring is unchanged.
- How it's inserted: `add_score()` inserts it in order and keeps the top 5. A tie goes below the existing equal score, so it never pushes an older one out. The file is only rewritten if the score made the table.
- What isn't recorded: losing a life, closing the window mid-run, or pressing R mid-run.
- How it's shown: the table appears on the CRASHED / YOU MADE IT overlay, over a dark panel, with "Final score" above it. Your run's row is highlighted green with `<- you`.

5. Checks I ran

All 10 persistence tests pass, and the Task 2 suite gives the same results as before.

- Persistence: the first run with no file, a new session loading old scores, and R restarts keeping the table.
- Ordering: top 5 only, sorted high to low, a win recorded the same way as a crash, and a non-qualifying score leaving the file untouched.
- Bad files: 11 corrupted variants (empty, garbage, truncated, wrong type, binary, deeply nested, mixed entries, too many entries) all start the game and are repaired at the next save.
- Save failures: the save path being a directory, or its folder missing, prints a warning, leaves no stray .tmp, and the game keeps running.
- Regression and soak: a drowning on the last life records correctly while earlier lives lost don't, and a soak of 216 runs across 6 sessions always left a valid, sorted file of at most 5 scores.

Things to know

- The score rewards survival time, not winning. The existing score is a survival timer. That means a fast win scores lower than a long run that ends in a crash, and the table treats them the same. I left it alone because you asked me not to replace the scoring. A win bonus would be an easy follow-up if you want it.
- The table only appears when a run ends. The HUD bar is already full, and a permanent panel would cover the road. I can add a toggle key if you want to see it during play.
- Hard to see in your video: scores are only visible after the first run ends, so for the "after" video, crash once and show the table, then restart and crash again to show it persisting. To show persistence across sessions, close the game and reopen it before the second crash.

Attached files: highscores.py, game_engine.py

Part 5 - Task 4: Day/Night Cycle

User

Now implement Task 4 ONLY: Day/Night Cycle.

Requirements:

- Automatically switch between day and night every 30 seconds.
- The timing must be based on elapsed time, not a fixed number of frames.
- The day/night cycle must continue while gameplay is active.
- Visually distinguish day mode from night mode.
- During night mode, cars must have headlights.
- The headlights should be visible farther ahead than the cars themselves to satisfy the requirement.
- The transition must not reset the score.
- The transition must not reset lives.
- The transition must not reset player position.
- The moving log/raft system must continue working.
- The persistent high-score system must continue working.
- Preserve all previous features from Tasks 1, 2 and 3.
- Do not remove existing gameplay.
- Keep the implementation compatible with the current Pygame project.
- Avoid unnecessary rewrites.

After implementation:

1. List all modified files.
2. Explain how the 30-second timer works.
3. Explain how night mode is rendered.
4. Explain how headlights are implemented.
5. Explain how you ensured previous tasks still work.
6. Check for obvious timing, rendering, collision, or state-reset bugs.

Claude

Process notes: Wiring day-night timing into engine reset and update calls. Before changing anything, I'll lock in the Task 3 state for diffing and look at the current draw order. Night rendering has to slot in after the scene but before the HUD, and the river band hides cars, so headlight drawing needs care there. All 21 earlier checks pass in both modes, with numbers identical to before. One last visual check: the game-over overlay and score table at night, since the table sits on top of the night tint.

Task 4 (the day/night cycle) is implemented and passes my headless tests, but I haven't played it in a visible window. I did render day, night and game-over frames to check how they look.

1. Files modified

- game/daynight.py (new): the 30-second timing function, the night tint, and the headlight drawing.
- game/game_engine.py: about 25 added lines: an import, three new fields in reset(), one line in update(), a small edit to the player-blink code in draw(), and a _draw_night() method.

Every other file is byte-identical to the Task 3 state. The new file uses CRLF like your others.

2. How the 30-second timer works

- reset() records cycle_start = pygame.time.get_ticks(), the real milliseconds on the clock.

- Each active `update()` calls `phase(get_ticks() - cycle_start)`, which returns `is_night = (elapsed // 30000) % 2 == 1`.
- The state is derived from the clock rather than counted up per frame, so the switch can't depend on frame rate. A test shows it landing at 30,000 and 60,000 ms at 5, 10, 30, 60, 144 and 500 FPS, even though the update counts range from 326 to 32,501.
- I also shrank the period to 300 ms and checked it against the real wall clock at a 60 FPS cap, and ran the real `run()` loop for about 1.6 seconds. Both flipped on schedule with no errors.
- Every run, including after R, starts in full daylight with a fresh 30-second clock. The clock only advances while the run is active, so it freezes with the game-over screen.
- The screen fades in and out over 1 second after each switch (`FADE_MS`). `is_night` itself flips at exactly 30.000 s.

3. How night mode is rendered

- Day is the original look, unchanged.
- At night, `_draw_night()` lays a dark blue tint over the road, cars, river and player, scaled by darkness. A road pixel goes from (60,60,60) to (28,31,51).
- The HUD is drawn after the tint, so it stays bright and readable. A test confirms the HUD strip is pixel-identical in day and night.
- The game-over overlay and score table sit on top of the tint and stay readable.

4. How headlights are implemented

`draw_headlights()` draws two lamps on the front of a car: a bright dot plus a cone that widens and fades with distance. It is pre-rendered once per direction and then reused.

- Direction: lamps go on the front edge, so cars driving down the screen light below themselves and cars driving up light above.
- Reach: the beam extends 170px past the nose, while a car is only 80px long. Tests confirm it is lit 100px and 125px ahead of the nose, and not lit in neighbouring lanes.
- Day: no lights are drawn at all.
- Fading: the lights scale with the night fade, so they never pop on or off against the screen.
- River: the beams are clipped out of the river band, because cars pass underneath it and a floating lamp over water would look like a bug.
- Player: I also gave the player's car headlights, since it is drawn as a car. They vanish while the car blinks after a hit.

5. How I made sure Tasks 1 to 3 still work

- Every file from Tasks 1 to 3 except `game_engine.py` is byte-identical to the previous state, and the engine diff only adds lines.
- I re-ran the full Task 2 and Task 3 suites (21 checks) normally and again with the game forced into permanent night, redrawing the full night scene every 7th update. All results are identical to before.
- Task 1's life behaviour is covered by the Task 2 suite's life checks and by the Task 4 tests at night. I didn't keep a separate Task 1 test file.

New tests cover the 30-second switch:

- Score, lives, player position, high scores and the log lane are unchanged across the switch.
- A log still carries the player at night, and falling still costs exactly 1 life.
- Game over at night saves the high score.
- R restarts in daylight.
- Soak: I ran 200,000 random-input frames with the period shrunk so every run saw several switches: 1,089 switches, 382 runs, 32,500 night frames, no errors. My first version of that soak never reached night, because random play crashes within about 8 seconds. I caught that and fixed it.
- Performance: a night frame with 14 cars costs about 4 ms against a 16.7 ms budget.

6. Bugs I looked for

- Timing: no frame-rate dependence, and exact boundaries at 30,000, 60,000 and 90,000 ms.
- State reset: none on the transition. The only reset is the intentional one on R.
- Collisions: unchanged, because night is purely visual.
- Rendering: the HUD, the river clip and the blink were all checked.
- Edge cases: the fade at the run start, and lights never drawn for hidden cars.

Things to know

- Your 10-second video can't show a 30-second switch. For recording only, you could temporarily set `DAY_NIGHT_MS = 5000` at the top of `daynight.py` (and maybe `FADE_MS = 500`). You must set it back to 30000 before committing, since that is what the task requires. Alternatively, record a longer clip and speed it up in editing.
- Existing behaviour I left alone: the score is still a survival timer, and the player can still drive off the bottom of the screen.
- All four tasks are now implemented. `README.md` is unchanged, so its checklist boxes are still for you to tick.

Attached files: `daynight.py`, `game_engine.py`

Appendix A - Files changed per task

Task	New files	Modified files
Task 1 Lives	-	game/game_engine.py, game/player.py
Task 2 Log/Raft	game/river.py	game/game_engine.py
Task 3 High scores	game/highscores.py	game/game_engine.py
Task 4 Day/Night	game/daynight.py	game/game_engine.py

main.py, traffic.py, __init__.py, requirements.txt and README.md were not changed. Each zip you uploaded holds the files for one task, but the game_engine.py inside every zip is the same final version (identical checksum), so the per-task engine snapshots are not recoverable from the zips.

Appendix B - Final source code of the changed files

game/game_engine.py (final, after Task 4)

```
1 import pygame
2 import random
3 from game.player import Player, LANE_W
4 from game.traffic import Car, make_car
5 from game.river import River, RIVER_TOP, RIVER_H
6 from game.highscores import load_scores, save_scores, add_score
7 from game.daynight import phase, draw_night, draw_headlights
8
9 LANES=8
10 WIDTH=LANES*LANE_W
11 HEIGHT=600
12 FPS=60
13 BG=(60,60,60)
14 START_LIVES=3
15 INVULN_FRAMES=90 # ~1.5s of protection after being hit
16
17 class GameEngine:
18     def __init__(self):
19         pygame.init()
20         self.screen=pygame.display.set_mode((WIDTH,HEIGHT))
21         pygame.display.set_caption("Traffic Escape")
22         self.clock=pygame.time.Clock()
23         self.font=pygame.font.SysFont("monospace",24,bold=True)
24         self.big_font=pygame.font.SysFont("monospace",44,bold=True)
25         self.high_scores=load_scores() # loaded once at start-up; survives R restarts
26         self.reset()
27
28     def reset(self):
29         self.start_pos=(WIDTH//2,HEIGHT-80)
30         self.player=Player(*self.start_pos)
31         self.lives=START_LIVES
32         self.invuln=0
33         self.river=River(WIDTH)
34         self.cars=[]
35         self.timer=0
36         self.spawn_interval=50
37         self.speed=3
38         self.score=0
39         self.game_over=False
40         self.won=False
41         self.score_rank=None # where this run's score landed in the table (None = not
42         in top 5)
43         self.cycle_start=pygame.time.get_ticks() # day/night clock restarts (in
44         daylight) with every new run
45         self.night=False
46         self.darkness=0.0 # 0 = full day, 1 = full night
47
48     def handle_events(self):
49         for event in pygame.event.get():
50             if event.type==pygame.QUIT: return False
51             if event.type==pygame.KEYDOWN and event.key==pygame.K_r: self.reset()
52         return True
53
54     def update(self):
55         if self.game_over or self.won: return
56         # day/night follows real elapsed milliseconds, so it doesn't depend on the
57         frame rate
58         self.night,self.darkness=phase(pygame.time.get_ticks()-self.cycle_start)
59         keys=pygame.key.get_pressed()
60         self.player.move(keys,0,WIDTH)
```

```

58     self.timer+=1
59     if self.timer>=self.spawn_interval:
60         lane=random.randint(0,LANES-1)
61         self.cars.append(make_car(lane,HEIGHT,self.speed))
62         self.timer=0
63         self.spawn_interval=max(22,self.spawn_interval-0.2)
64     if self.invuln>0: self.invuln-=1
65     for c in self.cars:
66         c.update()
67     # at most ONE life is lost per frame, and none while invulnerable
68     if self.invuln==0 and any(self.car_hits_player(c) for c in self.cars):
69         self.lose_life()
70     self.update_river()
71     self.cars=[c for c in self.cars if not c.off_screen(HEIGHT)]
72     self.score+=1
73     if self.score%300==0: self.speed=min(10,self.speed+0.5)
74     if self.player.rect.top<=10:
75         self.won=True
76     if self.game_over or self.won:
77         self.finish_run()
78
79     def car_hits_player(self,c):
80         # cars run underneath the river, so overlap that lies inside the river band
81         # doesn't count
82         if not c.rect.colliderect(self.player.rect): return False
83         o=c.rect.clip(self.player.rect)
84         return o.top<RIVER_TOP or o.bottom>RIVER_TOP+RIVER_H
85
86     def update_river(self):
87         if self.game_over: return
88         # the log the player is standing on (looked up before the logs move this
89         # frame)
90         log=self.river.log_at(self.player.rect.center)
91         self.river.update()
92         if log:
93             # ride the log, but never leave the screen sideways
94             r=self.player.rect
95             r.x=max(0,min(WIDTH-r.width,r.x+log.speed))
96             # over the water with no log underneath -> fell in (costs a life, no
97             # invulnerability exemption)
98             centre=self.player.rect.center
99             if self.river.contains(centre) and not self.river.log_at(centre):
100                 self.lose_life()
101
102     def lose_life(self):
103         self.lives-=1
104         if self.lives<=0:
105             self.lives=0
106             self.game_over=True
107         else:
108             self.player.respawn(*self.start_pos)
109             self.invuln=INVULN_FRAMES
110
111     def finish_run(self):
112         # runs exactly once per run: update() returns early on every frame after
113         # game_over / won
114         self.high_scores,self.score_rank=add_score(self.high_scores,self.score//10)
115         if self.score_rank is not None:
116             save_scores(self.high_scores)
117
118     def draw(self):
119         self.screen.fill(BG)
120         # road markings
121         for i in range(LANES+1):

```

```

pygame.draw.line(self.screen, (100,100,100), (i*LANE_W,0), (i*LANE_W,HEIGHT),2)
119     for y in range(0,HEIGHT,60):
120         for i in range(LANES):
121
pygame.draw.rect(self.screen, (200,200,100), pygame.Rect(i*LANE_W+LANE_W//2-
3,y,6,30))
122     # sidewalks
123     pygame.draw.rect(self.screen, (150,130,110), pygame.Rect(0, HEIGHT-50,WIDTH, 50))
124     pygame.draw.rect(self.screen, (150,130,110), pygame.Rect(0, 0,WIDTH, 30))
125     for c in self.cars: c.draw(self.screen)
126     self.river.draw(self.screen) # drawn over the cars: they pass underneath it
127     # blink the player while invulnerable after a hit
128     show_player=self.invuln==0 or (self.invuln//6)%2==0
129     if show_player:
130         self.player.draw(self.screen)
131     if self.darkness>0:
132         self._draw_night(show_player)
133     hud=pygame.Rect(0,0,WIDTH,30)
134     pygame.draw.rect(self.screen, (20,20,20), hud)
135     s=self.font.render(f"Score: {self.score//10}  GOAL: top!
R=Restart", True, (220,220,220))
136     self.screen.blit(s, (6,4))
137     lv=self.font.render(f"Lives: {self.lives}", True, (255,90,90))
138     self.screen.blit(lv, (WIDTH-lv.get_width()-6,4))
139     if self.game_over:
140         self._msg("CRASHED!", (220,60,60))
141     if self.won:
142         self._msg("YOU MADE IT!", (80,220,80))
143     if self.game_over or self.won:
144         self._draw_scores()
145     pygame.display.flip()
146
147 def _draw_night(self, show_player):
148     draw_night(self.screen, self.darkness)
149     # cars run underneath the river, so keep their lights out of the river band
150     for area in
(pygame.Rect(0,0,WIDTH,RIVER_TOP), pygame.Rect(0,RIVER_TOP+RIVER_H,WIDTH,HEIGHT))
:
151         self.screen.set_clip(area)
152         for c in self.cars:
153             draw_headlights(self.screen, c.rect, c.direction, self.darkness)
154         self.screen.set_clip(None)
155         if show_player:
156             draw_headlights(self.screen, self.player.rect, -1, self.darkness) # the
player drives up the screen
157
158 def _msg(self, text, color):
159     ov=pygame.Surface((WIDTH,HEIGHT), pygame.SRCALPHA)
160     ov.fill((0,0,0,150))
161     self.screen.blit(ov, (0,0))
162     m=self.big_font.render(text, True, color)
163     sub=self.font.render("Press R to Restart", True, (200,200,200))
164     self.screen.blit(m, (WIDTH//2-m.get_width()//2, HEIGHT//2-40))
165     self.screen.blit(sub, (WIDTH//2-sub.get_width()//2, HEIGHT//2+20))
166
167 def _draw_scores(self):
168     panel=pygame.Surface((360,228), pygame.SRCALPHA) # dark backing so the table
is easy to read
169     pygame.draw.rect(panel, (0,0,0,170), panel.get_rect(), border_radius=12)
170     self.screen.blit(panel, (WIDTH//2-180,348))
171     def line(text,y,color):
172         t=self.font.render(text, True, color)
173         self.screen.blit(t, (WIDTH//2-t.get_width()//2,y))
174     line(f"Final score: {self.score//10}", 356, (220,220,220))
175     line("HIGH SCORES", 396, (240,200,60))

```

```

176         for i,sc in enumerate(self.high_scores):
177             mine=(i==self.score_rank)
178             color=(80,220,80) if mine else (200,200,200)
179             t=self.font.render(f"{i+1}. {sc:>6}",True,color)
180             x=WIDTH//2-t.get_width()//2
181             y=428+i*28
182             self.screen.blit(t,(x,y))
183             if mine: self.screen.blit(self.font.render("<-
184             you",True,color),(x+t.get_width()+12,y))
185
186 def run(self):
187     running=True
188     while running:
189         running=self.handle_events()
190         self.update()
191         self.draw()
192         self.clock.tick(FPS)
193     pygame.quit()

```

game/player.py (Task 1)

```
1 import pygame
2
3 SPEED=4
4 LANE_W=80
5
6 class Player:
7     def __init__(self, x, y):
8         self.rect=pygame.Rect(x-20,y-30,40,60)
9         self.color=(60,160,220)
10        self.move_cooldown=0
11
12    def move(self, keys, min_x, max_x):
13        if self.move_cooldown>0:
14            self.move_cooldown-=1
15            return
16        dx=dy=0
17        if keys[pygame.K_LEFT] or keys[pygame.K_a]: dx=-LANE_W
18        if keys[pygame.K_RIGHT] or keys[pygame.K_d]: dx=LANE_W
19        if keys[pygame.K_UP] or keys[pygame.K_w]: dy=-8
20        if keys[pygame.K_DOWN] or keys[pygame.K_s]: dy=8
21        nx=max(min_x,min(max_x-self.rect.width,self.rect.x+dx))
22        ny=max(0,self.rect.y+dy)
23        if dx: self.rect.x=nx; self.move_cooldown=12
24        if dy: self.rect.y=ny
25
26    def respawn(self,x,y):
27        # put the player back at the start position (same centre as in __init__)
28        self.rect.center=(x,y)
29        self.move_cooldown=0
30
31    def draw(self,screen):
32        # car body
33        pygame.draw.rect(screen,self.color,self.rect,border_radius=8)
34        # windows
35        pygame.draw.rect(screen,(180,220,240),pygame.Rect(self.rect.x+6,self.rect.y+8,
36        28,18),border_radius=4)
37        # wheels
38        for wx in [self.rect.x+4,self.rect.right-12]:
39            for wy in [self.rect.y+4,self.rect.bottom-14]:
40
41                pygame.draw.rect(screen,(30,30,30),pygame.Rect(wx,wy,8,10),border_radius=3)
```

game/river.py (Task 2)

```
1 import pygame
2
3 RIVER_TOP=250      # y of the top edge of the river band
4 RIVER_H=80         # band height (one lane thick, taller than the 60px player)
5 LOG_W=140
6 LOG_COUNT=3
7 LOG_SPEED=2        # px per frame, positive = moves right
8 WATER=(40,90,160)
9 WAVE=(70,120,190)
10 LOG_COLOR=(140,95,50)
11 LOG_DARK=(100,65,30)
12
13 class Log:
14     def __init__(self,x,speed):
15         # the rect fills the whole band height, so "centre inside the rect" ==
16         # "standing on the log"
17         self.rect=pygame.Rect(x,RIVER_TOP,LOG_W,RIVER_H)
18         self.speed=speed
19
20     def update(self,width):
21         self.rect.x+=self.speed
22         cycle=width+LOG_W
23         # leave one side completely, then re-enter from the other (keeps the spacing
24         # constant)
25         if self.speed>0 and self.rect.left>=width: self.rect.x-=cycle
26         elif self.speed<0 and self.rect.right<=0: self.rect.x+=cycle
27
28     def draw(self,screen):
29         body=self.rect.inflate(0,-16)
30         pygame.draw.rect(screen,LOG_COLOR,body,border_radius=14)
31         pygame.draw.rect(screen,LOG_DARK,body,3,border_radius=14)
32         for gx in range(body.x+24,body.right-12,28):
33             pygame.draw.line(screen,LOG_DARK,(gx,body.y+10),(gx,body.bottom-10),2)
34
35 class River:
36     def __init__(self,width):
37         self.width=width
38         self.rect=pygame.Rect(0,RIVER_TOP,width,RIVER_H)
39         spacing=(width+LOG_W)//LOG_COUNT
40         self.logs=[Log(-LOG_W+i*spacing,LOG_SPEED) for i in range(LOG_COUNT)]
41
42     def update(self):
43         for log in self.logs: log.update(self.width)
44
45     def contains(self,point):
46         # True if the point is over the water band (log or not)
47         return self.rect.collidepoint(point)
48
49     def log_at(self,point):
50         # the log under the point, or None
51         for log in self.logs:
52             if log.rect.collidepoint(point): return log
53         return None
54
55     def draw(self,screen):
56         pygame.draw.rect(screen,WATER,self.rect)
57         for y in range(self.rect.top+14,self.rect.bottom,24):
58             for x in range(0,self.width,40):
59                 pygame.draw.line(screen,WAVE,(x,y),(x+16,y),2)
60         for log in self.logs: log.draw(screen)
```

game/highscores.py (Task 3)

```
1 import json
2 import os
3
4 MAX_SCORES=5
5 # Saved next to main.py (the project root), no matter which folder the game is started
   from.
6 SCORES_FILE=os.path.join(os.path.dirname(os.path.dirname(os.path.abspath(__file__))), "
   highscores.json")
7
8 def load_scores(path=SCORES_FILE):
9     """Return the saved scores, highest first (at most 5). Never raises:
10     a missing, unreadable or corrupted file just gives an empty / cleaned-up list."""
11     try:
12         with open(path,"r",encoding="utf-8") as f:
13             data=json.load(f)
14     except (OSError,ValueError,RecursionError):
15         return [] # first run (no file yet), unreadable file, or invalid JSON
16     if not isinstance(data,list):
17         return []
18     # keep only real non-negative whole numbers (bool is an int in Python, so exclude
       it)
19     scores=[s for s in data if isinstance(s,int) and not isinstance(s,bool) and s>=0]
20     return sorted(scores,reverse=True)[:MAX_SCORES]
21
22 def save_scores(scores,path=SCORES_FILE):
23     """Write the scores as a JSON list. Returns True on success; a failure never
       crashes the game."""
24     tmp=path+".tmp"
25     try:
26         with open(tmp,"w",encoding="utf-8") as f:
27             json.dump(scores,f)
28         os.replace(tmp,path) # swap in the finished file, so an interrupted write
       can't damage the old one
29         return True
30     except OSError as err:
31         print("Could not save high scores:",err)
32         try: os.remove(tmp)
33         except OSError: pass
34         return False
35
36 def add_score(scores,score):
37     """Insert score into the sorted (highest first) list.
38     Returns (new_list_of_at_most_5,
       rank_index_of_the_new_score_or_None_if_it_didn't_make_it)."""
39     rank=sum(1 for s in scores if s>=score) # ties go below existing equal scores
40     if rank>=MAX_SCORES:
41         return scores,None
42     return (scores[:rank]+[score]+scores[rank:][:MAX_SCORES],rank)
43
```

game/daynight.py (Task 4)

```
1 import pygame
2
3 DAY_NIGHT_MS=30000    # switch between day and night every 30 seconds (real time, not
                        # frames)
4 FADE_MS=1000          # the screen fades to / from night over this long after each
                        # switch
5 NIGHT_TINT=(5,10,45)
6 NIGHT_ALPHA=145       # how dark the overlay is at full night (0-255)
7 BEAM_LEN=170          # headlight beam reaches this far past a car's front (a car is
                        # only 80px long)
8 LIGHT_W=56            # width of the glow around one headlight
9 LAMP_PAD=10           # rows of the light image that sit inside the car's nose
10
11 def phase(elapsed_ms):
12     """Return (is_night, darkness) for the milliseconds elapsed since the run started.
13     is_night flips exactly every DAY_NIGHT_MS. darkness (0 = day .. 1 = night) eases
14     over FADE_MS."""
15     is_night=(elapsed_ms//DAY_NIGHT_MS)%2==1
16     fade=min(1.0,(elapsed_ms%DAY_NIGHT_MS)/FADE_MS)
17     if is_night: return True,fade
18     if elapsed_ms<DAY_NIGHT_MS: return False,0.0 # a run always starts in full
19     daylight
20     return False,1.0-fade
21
22 def draw_night(screen,darkness):
23     # dark blue tint over the whole scene (the HUD is drawn afterwards, so it stays
24     # readable)
25     ov=pygame.Surface(screen.get_size(),pygame.SRCALPHA)
26     ov.fill((*NIGHT_TINT,int(NIGHT_ALPHA*darkness)))
27     screen.blit(ov,(0,0))
28
29 _lights={}
30
31 def _light(direction):
32     # one lamp = a bright dot plus a cone that widens and fades with distance; built
33     # once, then reused
34     if not _lights:
35         down=pygame.Surface((LIGHT_W,LAMP_PAD+BEAM_LEN),pygame.SRCALPHA)
36         mid=LIGHT_W//2
37         for i in range(BEAM_LEN):
38             t=i/BEAM_LEN
39             half=int(7+17*t)
40             alpha=int(120*(1-t)**1.3)
41             pygame.draw.line(down,(255,240,170,alpha),(mid-
42             half,LAMP_PAD+i),(mid+half,LAMP_PAD+i))
43             pygame.draw.circle(down,(255,250,200),(mid,LAMP_PAD-5),5)
44         _lights[1]=down # heading down the
45         screen
46         _lights[-1]=pygame.transform.flip(down,False,True) # heading up the screen
47     return _lights[direction]
48
49 def draw_headlights(screen,rect,direction,strength=1.0):
50     """Two headlights on the front of a car whose rect is `rect`, heading `direction`
51     (1=down, -1=up).
52     strength (0..1) scales the brightness so the lights fade in and out with the
53     night."""
54     img=_light(direction)
55     img.set_alpha(int(255*strength))
56     if direction>0: top=rect.bottom-LAMP_PAD # front edge is the bottom
57     else: top=rect.top-BEAM_LEN # front edge is the top
58     inset=max(6,rect.width//4)
59     for lx in (rect.left+inset,rect.right-inset):
60         screen.blit(img,(lx-LIGHT_W//2,top))
```

